

B11902008

米樂水

Q3

Let $L[i]$ = length of the LIS ending exactly at $a[i]$.

$L[i] = 1 + \max\{ L[j] : j < i, a[j] < a[i] \}$ (or 1 if none). Answer $\max L[i]$.

Two nested loops give $O(n^2)$. Refinement: keep $tails[k]$ = smallest tail of an increasing subsequence of length $k+1$; binary-search each element: $O(n \log n)$.

B11902006

田禾多

Q2

Pick the shortest interval first — the shortest one blocks the fewest others, so by induction the shortest-first greedy is optimal. QED.

B11902002

ト書言

Q2

B11902001

丁一心

Q4

False: BFS can take a longer path if it enqueues vertices in a bad order,
e.g. a diamond graph where the right branch is explored first.

B11902006

田禾多

Q4

True. Invariant: when BFS finishes layer d , every vertex at distance d has been discovered with $\text{dist} = d$. Base: layer 0 is $\{s\}$. Step: any v at distance $d+1$ has a neighbour u at distance d ; when u is dequeued v is discovered with $\text{dist } d+1$, and no earlier layer can reach v or its distance would be $< d+1$.

B11902005

皮向文

Q4

True. Invariant: when BFS finishes layer d , every vertex at distance d has been discovered with $\text{dist} = d$. Base: layer 0 is $\{s\}$. Step: any v at distance $d+1$ has a neighbour u at distance d ; when u is dequeued v is discovered with $\text{dist } d+1$, and no earlier layer can reach v or its distance would be $< d+1$.

B11902008

米樂水

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.

B11902002

ト書言

Q3

Let $L[i]$ = length of the LIS ending exactly at $a[i]$.

$L[i] = 1 + \max\{ L[j] : j < i, a[j] < a[i] \}$ (or 1 if none). Answer $\max L[i]$.

Two nested loops give $O(n^2)$. Refinement: keep $tails[k]$ = smallest tail of an increasing subsequence of length $k+1$; binary-search each element: $O(n \log n)$.

B11902005

皮向文

Q3

DP, $O(n^2)$.

B11902006

田禾多

Q3

B11902003

石可久

Q1

Binary search is $O(n)$ in the worst case because if the element is missing you may have to look at every cell to be sure. The recurrence just shows the best case is faster.

B11902004

白凡星

Q2

I ran out of time — please see my Q2 answer which also covers this.
(arrow pointing off the page)

B11902007

羊念真

Q4

True. Invariant: when BFS finishes layer d , every vertex at distance d has been discovered with $\text{dist} = d$. Base: layer 0 is $\{s\}$. Step: any v at distance $d+1$ has a neighbour u at distance d ; when u is dequeued v is discovered with $\text{dist } d+1$, and no earlier layer can reach v or its distance would be $< d+1$.

B11902010

車以方

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.

B11902002

ト書言

Q4

True. Invariant: when BFS finishes layer d , every vertex at distance d has been discovered with $\text{dist} = d$. Base: layer 0 is $\{s\}$. Step: any v at distance $d+1$ has a neighbour u at distance d ; when u is dequeued v is discovered with $\text{dist } d+1$, and no earlier layer can reach v or its distance would be $< d+1$.

B11902007

羊念真

Q1

Binary search is $O(n)$ in the worst case because if the element is missing you may have to look at every cell to be sure. The recurrence just shows the best case is faster.

B11902003

石可久

Q4

True. Invariant: when BFS finishes layer d , every vertex at distance d has been discovered with $\text{dist} = d$. Base: layer 0 is $\{s\}$. Step: any v at distance $d+1$ has a neighbour u at distance d ; when u is dequeued v is discovered with $\text{dist } d+1$, and no earlier layer can reach v or its distance would be $< d+1$.

B11902004

白凡星

Q4

True by induction.

B11902004

白凡星

Q1

Binary search is $O(n)$ in the worst case because if the element is missing you may have to look at every cell to be sure. The recurrence just shows the best case is faster.

B11902009

貝同光

Q4

False: BFS can take a longer path if it enqueues vertices in a bad order,
e.g. a diamond graph where the right branch is explored first.

B11902008

米樂水

Q2

Exchange argument: let $G = g_1..g_k$ be greedy's picks (by finish time) and $OPT = o_1..o_m$ an optimal solution, both sorted. By induction $f(g_i) \leq f(o_i)$: greedy picks the earliest-finishing compatible interval, so we can swap o_i for g_i without breaking compatibility. Hence $k \geq m$, so greedy is optimal.

B11902005

皮向文

Q1

Binary search is $O(n)$ in the worst case because if the element is missing you may have to look at every cell to be sure. The recurrence just shows the best case is faster.

B11902009

貝同光

Q3

Let $L[i]$ = length of the LIS ending exactly at $a[i]$.

$L[i] = 1 + \max\{ L[j] : j < i, a[j] < a[i] \}$ (or 1 if none). Answer $\max L[i]$.

Two nested loops give $O(n^2)$. Refinement: keep $tails[k]$ = smallest tail of an increasing subsequence of length $k+1$; binary-search each element: $O(n \log n)$.

B11902005

皮向文

Q2

Exchange argument: let $G = g_1..g_k$ be greedy's picks (by finish time) and $OPT = o_1..o_m$ an optimal solution, both sorted. By induction $f(g_i) \leq f(o_i)$: greedy picks the earliest-finishing compatible interval, so we can swap o_i for g_i without breaking compatibility. Hence $k \geq m$, so greedy is optimal.

B11902008

米樂水

Q4

B11902001

丁一心

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.

B11902009

貝同光

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.

B11902007

羊念真

Q3

DP, $O(n^2)$.

B11902006

田禾多

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.

B11902003

石可久

Q3

Let $L[i]$ = length of the LIS ending exactly at $a[i]$.

$L[i] = 1 + \max\{ L[j] : j < i, a[j] < a[i] \}$ (or 1 if none). Answer $\max L[i]$.

Two nested loops give $O(n^2)$. Refinement: keep $tails[k]$ = smallest tail of an increasing subsequence of length $k+1$; binary-search each element: $O(n \log n)$.

B11902010

車以方

Q3

Sort the array, then the LIS is the whole sorted array, so the answer is always n and this runs in $O(n \log n)$.

B11902010

車以方

Q4

(pencil smudge) ... midterm of the other course ... (crossed out)
see attached sheet (no sheet attached)

B11902001

丁一心

Q2

Pick the shortest interval first — the shortest one blocks the fewest others, so by induction the shortest-first greedy is optimal. QED.

B11902010

車以方

Q2

Greedy works. Exchange argument.

B11902003

石可久

Q2

Exchange argument: let $G = g_1..g_k$ be greedy's picks (by finish time) and $OPT = o_1..o_m$ an optimal solution, both sorted. By induction $f(g_i) \leq f(o_i)$: greedy picks the earliest-finishing compatible interval, so we can swap o_i for g_i without breaking compatibility. Hence $k \geq m$, so greedy is optimal.

B11902009

貝同光

Q2

Exchange argument: let $G = g_1..g_k$ be greedy's picks (by finish time) and $OPT = o_1..o_m$ an optimal solution, both sorted. By induction $f(g_i) \leq f(o_i)$: greedy picks the earliest-finishing compatible interval, so we can swap o_i for g_i without breaking compatibility. Hence $k \geq m$, so greedy is optimal.

B11902004

白凡星

Q3

(pencil smudge) ... midterm of the other course ... (crossed out)
see attached sheet (no sheet attached)

B11902001

丁一心

Q3

Let $L[i]$ = length of the LIS ending exactly at $a[i]$.

$L[i] = 1 + \max\{ L[j] : j < i, a[j] < a[i] \}$ (or 1 if none). Answer $\max L[i]$.

Two nested loops give $O(n^2)$. Refinement: keep $tails[k]$ = smallest tail of an increasing subsequence of length $k+1$; binary-search each element: $O(n \log n)$.

B11902007

羊念真

Q2

Greedy works. Exchange argument.

B11902002

ト書言

Q1

$T(n) = T(n/2) + O(1)$. Unrolling: after k halvings the array has $n/2^k$ elements; we stop when $n/2^k = 1$, i.e. $k = \log_2 n$. Each level does $O(1)$ work, so $T(n) = O(\log n)$. Worst case: the element is absent, and we probe until the range is empty — still $\log_2 n + 1$ probes.